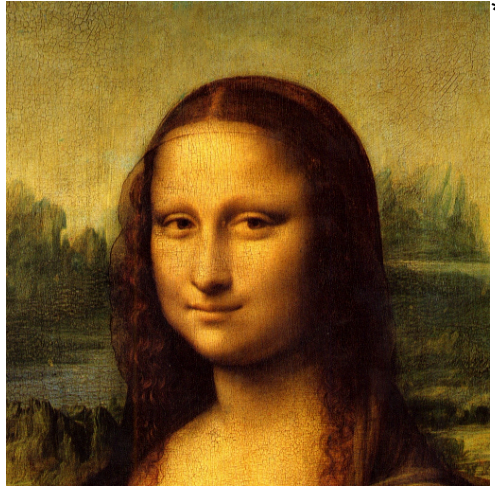




Project Report

Sean Larsen - seala24@student.sdu.dk

Kristóf Ákos Koltai - akkol24@student.sdu.dk

Denis Nedzhibov - dened24@student.sdu.dk

Igor Zimmermann - igzim24@student.sdu.dk

Örs Tomaj Jeney - orjen24@student.sdu.dk

Dávid Borka - dabor24@student.sdu.dk

* The Mona Lisa is considered as Da Vinci's magnum opus.
This project is our magnum opus.

Contents

1. Abstract	4
2. Introduction	5
2.1. Motivation	5
2.2. Objective	5
3. Methodology	7
3.1. Task tracking	7
3.2. Documents and Presentations	7
3.3. Repository	7
4. Problem analysis	9
4.1. Background and Motivation	9
4.2. Problem Statement	9
4.3. Aim	10
4.4. Use cases	10
4.4.1. Booking process	10
4.4.2. Consultation process	11
4.4.3. Review and send doctor's note to patient	11
4.5. Risks	11
4.5.1. Security	11
4.5.2. Componentised Architecture	12
5. Requirements	13
5.1. Functional Requirements	13
5.2. Non-Functional Requirements	14
6. Design	16
6.1. System architecture	16
6.1.1. Application layer diagram	16
6.1.2. Two monolithic backend approaches	16
6.1.3. Component-Based System Diagram	16
6.2. Tech stack	17
6.3. Frontend	17
6.4. Backend architecture	18
6.4.1. Tech stack justification	18
6.4.2. Booking backend as proof of concept	18
6.4.3. Controller separation	18
6.4.4. Service and infrastructure layers	18
6.4.5. Data Transfer Objects	19
6.4.6. Error handling	19
6.4.7. Authentication and authorization	19
6.4.8. API endpoints	19
6.5. Database design	20
6.5.1. Relational database	21
6.5.2. Non-relational database	21

7. Implementation	22
7.1. Naming conventions	22
7.2. Containerization	22
7.2.1. Startup Order & Healthchecks	22
7.2.2. Environment Variables	22
7.2.3. Persistent Volumes	23
7.3. Service-by-Service Implementation	23
7.3.1. Speech-to-Text Service (Echo)	23
7.3.2. LLM Service (Odin)	23
7.3.3. PDF Generation Service (Saga)	23
7.3.4. Email Service (Hermes)	23
7.3.5. Booking Backend (Janus)	23
7.3.6. Consultation Backend (Heimdall)	24
7.3.7. Booking Frontend (Iris)	24
7.3.8. Consultation Frontend (Eir)	24
7.4. Inter-Service Implementation	25
7.5. Authentication & Authorization	25
8. Validation	26
8.1. Testing	26
8.1.1. Consultation Backend - ASP.NET	26
8.1.2. Booking Backend - ASP.NET	27
8.1.3. Test Coverage and Results for Backends	28
8.1.4. LLM Service - Liquid AI ¹	28
8.1.5. Speech-to-text Service - Faster-Whisper ²	29
8.1.6. Manual Testing	31
8.2. Supporting Technologies	32
8.2.1. Auth0 for Authentication ³	32
8.2.2. Mailpit for Email Testing ⁴	32
8.2.3. Typst for PDF Generation ⁵	32
8.2.4. Docker and Docker Compose	32
9. Conclusion	33
9.1. Summary	33
9.2. Future work	33
Appendix	35

¹docs/research/LLM-testing.typ

²docs/research/STT-test.typ

³docs/research/authentication.typ

⁴docs/research/email.typ

⁵docs/research/pdf.typ

1. Abstract

This report addresses the administrative burden faced by outpatient departments, where Danish GPs consult an average of 49 patients per day while spending 10-20 hours per week on administrative tasks. To tackle this, a privacy-preserving outpatient department management system was developed, keeping all sensitive patient data within the clinic's network. The motivation is to modernize clinical workflows using open-source AI tools, improve consultation quality through AI-assisted second opinions, and apply the combined knowledge from this semester's courses.

The objectives were to deliver a working consultation management system that automates the workflow from audio recording to prescription delivery, while providing doctors with AI-generated summaries and medical suggestions, and allowing patients to manage their bookings.

In a task-driven development environment, the system was developed as two separate web applications backed by locally hosted speech recognition and language models for transcription and document generation, with secure authentication and a consistent deployment environment across all services.

The finished system covers the full consultation workflow, with separate access for doctors and patients, AI-generated summaries and prescriptions that doctors can review and adjust, and automatic prescription delivery to patients by email.

By the end of the semester, a working and secure outpatient consultation system had been built, putting into practice a service-based architecture alongside locally hosted AI. The system can be extended further, for example by incorporating patient history into AI suggestions, processing multiple audio recordings simultaneously, or expanding the AI's role in supporting diagnosis.

2. Introduction

Outpatient departments face an ever-increasing load and pressure to maintain high patient volumes while maintaining quality patient care, often relying on outdated systems.

This report documents the work of this semester's project, a privacy-preserving OPD (Outpatient Department) management system. The project applies skills gathered from this semester's subjects to provide a working solution for managing consultations in clinical environments, from appointment bookings to AI-assisted prescriptions.

The system consists of a web application that lets patients handle appointments and allows doctors full control over the consultation workflow that can be broken down into the following: audio recording, transcription, AI-generated summary, AI-generated medical suggestions, and finally, delivery of the doctor's note via email to the patient.

This project follows a component-based architecture, where components are connected through well-defined interfaces for an easily maintainable system. Additionally, to ensure better data security and privacy, the AI models are hosted locally to guarantee that no patient data ever leaves the clinic's network.

2.1. Motivation

Danish GPs see an average of 49 patients per day, leaving little to no time for patient care due to the administrative overhead. Managing appointments on paper, taking notes during consultations, and writing prescriptions in separate tools, all contributing to staff burnout. As a consequence, patients suffer from longer wait times, shorter consultation time due to lack of time and doctors losing work hours on paperwork.⁶

To address these problems, AI-based tools can be used to ease the workload by automating the creation of prescriptions, managing doctor's appointments and providing AI-suggested second opinions. Given the sensitivity of medical records / health data and the strict data protection requirements, all of these tools are locally hosted, ensuring patient data never leaves the clinic's network. This was a priority throughout the design and implementation of this project.

2.2. Objective

The project's aim is to deliver a working OPD management system that supports doctors' everyday workflows. The system's design provides doctors with a user-friendly interface to record and manage consultations and easily create prescriptions along with AI suggested medical advice.

The booking system serves as a proof of concept to simulate how a full patient-to-doctor workflow would look like in a production like environment. This also provides

⁶Beskrivelse af almen praksissektoren i Danmark (2016)

the system with the patient's email address needed to send the doctors note to complete the full consultation workflow.

To achieve this, the project provides the following core capabilities:

Appointment Management: Patients are able to register, log in, choose from doctors, and create bookings.

Audio Recording and Transcription: Doctors begin and conclude consultations from their portal. The system records the audio and passes it to a local speech-to-text service for transcription.

Summary Generation: A locally hosted LLM, served through Ollama, processes the transcript and produces a structured consultation summary to cut down irrelevant non-medical transcript. The summary is then manually checked by the doctor for accuracy before finalization.

Prescription Review and Delivery: Upon summary validation, a prescription is generated along with AI-generated suggestions such as overlooked medications or referrals that act as a second opinion. Doctors can then revise the prescription if needed and finally send a formatted PDF to patients via email.

This report covers the methodology, the problem analysis and the requirements to set boundaries and limitations to our objective, followed by the implementation and design of each component to achieve this goal. The report concludes with the testing and validation process used to verify the set limitations and functions.

3. Methodology

Before starting out on our project, we outlined a set of rules and methods that ensured that progression throughout the project would be smooth and continuous. The methodology and tools used ensured that every contribution made to the project was peer-reviewed which added accountability to each task.

3.1. Task tracking

In order to ensure all of the tasks are accounted for, we made every task (programming, diagramming, documenting) an issue on Jira. We completed these tasks during one-week-long Sprints, from one Monday to the next. Some exceptions were made with the length of the Sprints, for example around the spring break. To see the velocity report chart of this project, see Appendix Figure B.

On Mondays, we held meetings, where we reflected on the Sprint ending that day and planned the one coming up by assigning tasks from the backlog. On Thursdays, we held stand-up meetings, where everyone gave an update on their issue(s) to track each others progress, and to find bottlenecks as soon as possible.

To ensure that tasks were distributed fairly, we assigned tasks not based on their sheer quantity, but on actual difficulty. We agreed collectively on an issue's story point value using Story Point Poker, see example Appendix Figure C.

3.2. Documents and Presentations

To create our documentation and presentations, we chose Typst. Typst is a text-based document markup language that allowed us to version control and handle our documents as if they were code. We created templates to achieve a uniform look for all of our reports and presentations. In order to keep track of our images and diagrams, we have also decided to store them in the same repository as our documents and code. We have created a meeting log document during each meeting to keep everyone accountable and to allow team members to catch up in case they were absent.

3.3. Repository

To ensure code contributions are safe, each issue had its own branch, and pull requests had to be opened. Those pull requests had to be reviewed by at least two non-contributing members. In order to allow for a rigorous and in-depth review of each contribution, we made it a rule that pull requests must be opened by Fridays, which left us the entire weekend for review and refactoring.

To support reviewers and to guarantee a smooth workflow, a Pull Request Template has also been made, see Appendix Figure A. We merged all pull requests together during our Monday meetings, so we could resolve possible merge conflicts with all contributors' inputs. We have created a pipeline that sends a message to our Discord

server about a new pull request, and by replying to that message, the PR owner had to tag the requested reviewers.

All in all, these methods and rules ensured consistent progress across all sprints and an even distribution of work with no missed deadlines, compared to previous semesters. With constant peer reviewing, our codebase remained consistent and maintained quality.

4. Problem analysis

4.1. Background and Motivation

As mentioned in the introduction, outpatient departments face outdated manual processes that drastically slow down operations and put a burden on both patients and medical staff. One main cause of this problem is due to the legacy IT infrastructure. Hospitals run old and outdated systems that are expensive and risky to replace during their constant operation. Additionally, integrating new clinical tools require large amounts of resources to transfer data, train staff, and comply with data privacy compliances. This makes moving on from legacy systems costly and slow. Moreover, these old legacy systems are a large threat to cyber-attacks yet they contain highly sensitive patient health data.

On top of that, GPs and physicians report spending between 10 and 20 hours per week on administrative tasks alone.⁷ This large overhead on top of the stress of patient care causes large diagnostic errors, with 58% of them occurring during GP consultation.⁸

Commercial health systems, like Epic, do exist and are widely used, however they are often cloud based. Sending patient data to external servers to process with AI tools creates a large GDPR compliance issue in case of a data breach.⁹

This highlights a need for a solution: simple AI tools that run entirely locally using open-source models. Keeping all sensitive health information secure within the clinic's network, relieve medical staff of administrative work and modernize facilities quickly and affordably.

4.2. Problem Statement

Danish GPs consult with around 49 patients per day on average.¹⁰ Operating at this quantity and volume leaves little room for error, yet GPs and physicians report spending between 10 and 20 hours per week on administrative tasks alone.¹¹ This administrative overhead impacts doctor to patient care quality, with 58% of diagnostic errors occurring during GP consultations in OPD environments.¹² and prescription errors ranging from 1% to 11% of all prescriptions written.¹³

⁷ Medscape Physician Compensation Report (2018). American Medical Association.

⁸ Patient Claim Line, Medical Misdiagnosis Statistics (2024). <https://www.patientclaimline.com/article/medical-misdiagnosis-statistics/>

⁹ GDPR Register, Navigating GDPR in Healthcare. <https://gdprregister.eu/gdpr/healthcare-sector-gdpr>

¹⁰ Beskrivelse af almen praksissektoren i Danmark (2016)

¹¹ Medscape Physician Compensation Report (2018). American Medical Association.

¹² Patient Claim Line, Medical Misdiagnosis Statistics (2024). <https://www.patientclaimline.com/article/medical-misdiagnosis-statistics/>

¹³ Wikipedia, Medical Error. https://en.wikipedia.org/wiki/Medical_error

These statistics clearly highlight a problem and a problem to solve. The current situation on administrative workload contributes to mistakes and affect patient care outcomes in diagnosis and prescribing.

4.3. Aim

The aim of this project is to provide a privacy-preserving, locally hosted OPD management system that automates the workflow from consultation to digital prescription, using open-source speech-to-text and large language models, with a “safety net” of suggestive alerts.

The aim is built on two main technical focuses: component-based architecture and privacy first design. Managing such workflow in a single, tightly coupled system would make it difficult to maintain, extend and replace individual parts. At the same time, dealing with health data requires strict precautions, meaning only locally hosted open-source models are used, so that patient data never leaves the hospital's network.

To achieve this, the system is broken down into multiple independent services: appointment management, transcription, clinical summarisation, medical suggestions, and prescription generation. These services are isolated, each with its own runtime environment and well-defined interfaces that connect them and enable easy replacement and seamless upgrades of individual components without affecting the rest of the system.

4.4. Use cases

The following use cases follow the two primary user workflows: the doctor's workflow for a consultation broken down into two segments, consultation and then review. Additionally, a booking workflow is included as part of the proof of concept. The following use cases outline the system behaviour and actor interactions.

4.4.1. Booking process

- **Primary actor:** Patient
- **Preconditions:** Patient is registered in the system and has valid login credentials
- **Goal:** Book a consultation with a doctor

Flow (see *Appendix Figure D*, *Appendix Figure E*)

1. The user navigates to the client website and logs in
2. The user selects a preferred doctor
3. The system displays available time slots
4. The user books an available appointment
5. The user is redirected to a dashboard where they can view their bookings
6. The system sends a confirmation email

4.4.2. Consultation process

- **Primary actor:** Doctor
- **Secondary actor:** Patient
- **Preconditions:** An appointment exists in the system and the doctor has valid credentials
- **Goal**
 - Doctor and patient discuss their medical concerns
 - Doctor identifies symptoms, makes a diagnosis, and if necessary prescribes medicine and refers the patient to another department
 - The system internally processes the audio and transcribes the consultation

Flow (see *Appendix Figure F*)

1. Doctor navigates to the consultation website and logs in
2. The system displays a dashboard with the day's appointments for the doctor
3. The user selects the appropriate appointment and starts consultation
4. The system begins audio recording
5. The system waits until the user ends the consultation
6. Audio recording is stopped and sent to the speech-to-text component for transcription
7. The Doctor validates transcription and can update any mistakes or missing information

4.4.3. Review and send doctor's note to patient

- **Primary actor:** Doctor
- **Preconditions:** Consultation recording completed and successfully transcribed
- **Goal:** Review AI-generated documentation, validate clinical suggestions, and send the final doctor's note to the patient

Flow (see *Appendix Figure F*)

1. The system generates a summary based on the transcription and displays it to the doctor
2. The doctor can either edit or accept the summary
3. When the summary is accepted, the system generates a draft prescription along with suggestions
4. The doctor reviews, edits and approves the final prescription
5. The system exports the prescription as a PDF and emails it to the patient

4.5. Risks

4.5.1. Security

The most significant risk in this project is the mishandling of sensitive patient data. To avoid this, the system is split into two separate locally hosted backends: a booking service and a clinical workflow service. By keeping them separate, the booking backend has zero access to clinical data, meaning even if the booking system is compromised, patient medical records remain unreachable.

As this system handles sensitive EU patient health data, security was a priority throughout the design and implementation of this project. To reduce the risk of data breaches, all AI models are locally hosted ensuring no patient data is sent to external servers.

Structured booking data is stored in a relational database, while sensitive clinical data (transcripts, summaries, and prescriptions) is stored in a separate non-relational database. Role-based access control and token-based authentication prevent unauthorised access at the backend level, with doctors and patients having separate authentication tenants and access scopes. Data in transit is encrypted in non-development environments to further protect sensitive information.

4.5.2. Componentised Architecture

A tightly coupled system would make individual failures spread across the entire application. By breaking the system into loosely coupled components with clearly defined interfaces, failures are isolated to the affected component while the rest of the system continues to function. For example, if the LLM service goes down, only the summary and prescription generation is affected while the rest of the system continues to operate. This also simplifies debugging and makes individual components independently replaceable without affecting others.

Docker containerisation is used to enforce isolation between services. Each component runs in its own container, which also enables independent scaling and automatic health checks. Containers that fail to respond can be automatically restarted, meaning the system recovers from individual failures without any manual intervention, directly addressing the reliability requirements of the system.

5. Requirements

5.1. Functional Requirements

MUST have

- The system shall allow patients to create and view bookings.
 - Acceptance criteria:
 - Creating a booking returns a booking identifier and status.
- The system shall require authenticated access for patient and doctor workflows.
 - Acceptance criteria:
 - Protected API endpoints reject unauthenticated requests.
 - Authenticated users can access only authorized workflow endpoints.
- The system shall allow doctors to start and end consultations linked to valid bookings.
 - Acceptance criteria:
 - Start creates a consultation record with status *In Progress*.
 - End sets consultation status to *Completed*.
- The system shall transcribe consultation audio locally using an open-source model.
 - Acceptance criteria:
 - Uploading audio returns transcript text.
 - No cloud Speech-to-Text service is called during transcription.
- The system shall generate a structured clinical summary and prescription draft using a locally hosted open-source LLM.
 - Acceptance criteria:
 - Summary generation returns structured summary output.
 - Prescription draft generation returns structured fields.
- Doctors shall be able to review, edit, approve, and reject AI-generated summary and prescription outputs.
 - Acceptance criteria:
 - Edit operations persist updated content.
 - Approve operation marks status Approved.
 - Reject operation marks status Rejected.
- The system shall generate a PDF prescription after doctor approval and send it to the patient via email.
 - Acceptance criteria:
 - Approval triggers PDF generation.
 - Approved PDF is attached to outbound email.
 - Delivery outcome is logged.
- The system shall store structured operational data (users, doctors, patients, bookings) in a relational database.

- The system shall store transcript and AI-generated consultation artifacts in a non-relational database.

SHOULD have

- The system should allow searching and filtering doctors by availability and name/specialty.
- The system should log AI prompts and outputs for evaluation purposes.

COULD have

- Doctors could upload diagnostic images to consultation records.
- The system could support runtime switching between local LLM models.
- The system could allow configurable instructions.

WON'T have

- The system will not use cloud-based AI services for core transcription or generation.
- The system will not include a mobile application.

5.2. Non-Functional Requirements

Performance

- Transcription request returns within 60 seconds for a 5-minute audio sample on low-end hardware.
- PDF generation and email dispatch complete within 5 seconds after approval under normal load (up to 10 concurrent users).

Portability

- The system runs on Windows, Linux, and macOS through containerized deployment.
- All core services are runnable with one orchestrated compose configuration.

Scalability / Capacity

- The system supports at least 5 concurrent consultations without service failure.

Maintainability / Modularity

- Core services are independently deployable and replaceable via stable APIs.
- The architecture follows CBSE principles with clear component boundaries and interfaces.
- Changes to one service should not require code changes in unrelated services.

Security

- Passwords and secrets shall never be stored in plain text.
- Role-based authorization rules shall enforce doctor and patient access boundaries.
- Security events (login failure, unauthorized access, approval actions) shall be auditable.
- JWT-based token sessions shall be created and managed by Auth0, protecting private pages

Reliability / Recoverability

- No approved prescription or transcript shall be lost across restart events.
- Relational and non-relational stores shall use persistent volumes.

Testability

- Core backend logic shall include unit and integration tests.
- At least 80% coverage shall be achieved for core application logic.

6. Design

6.1. System architecture

This semester's project was designed with a component-based system (CBS) design approach focusing on modularity and reusability, communicating through well-defined interfaces via RESTful APIs, where components are independently deployed, enabling fault isolation and easy replaceability.

A layered architecture enforces vertical separation of concerns across frontend, backend, and databases.

6.1.1. Application layer diagram

The system is structured into three main layers: frontend, backend, and services/databases, as illustrated in Appendix Figure G.

Two frontends exist - one for the consultation system and one as a proof-of-concept for the booking system. Each frontend communicates only with the corresponding backend via RESTful API calls.

The backends act as orchestrators, handling business logic and using the independent infrastructure to off-load tasks such as PDF generation and LLM prompting. Additionally, two separate databases exist for relational booking data and a non-relational for patient health data.

6.1.2. Two monolithic backend approaches

At an early stage, a microservices architecture was evaluated as an alternative to a monolithic approach. Although a microservices architecture would align with the component-based design goals, the overall system scope and operational complexity made a monolithic structure a better fit for this implementation.

In addition, two separate backends were defined for the two primary use cases - main consultation backend and a proof-of-concept booking backend. This separation improves reliability, since a failure in one backend does not affect the availability of the other. Additionally, by keeping the two systems separate, the booking system is independent and isolated from any connection to sensitive patient data and allows the doctors consultation backend to be hosted locally.

6.1.3. Component-Based System Diagram

The Component-Based System (CBS), see Appendix Figure H, illustrates how the system is structured into modular components with explicit provided and required interfaces.

The two backends act as the central orchestrators:

- Booking Service: exposes IAuthentication, IBooking and IAvailability interfaces depending on IRelationalDb and IEmailService
- Consultation Service: exposes ITranscription, ISummaryEditor and ISendPrescription among others mentioned previously, depending on all interfaces.

Using a component-based design, with dependencies only on the interfaces allows for interchangeable services without modifying the consuming component.

6.2. Tech stack

The choices were made with the following in mind: familiarity with the language, framework or model over technical suitability reduces the learning curve. However, we did research for each layer, and each member evaluated and made a choice based on which technology fits our group and project the best. For transcription, Faster-Whisper came out as the most reliable in testing, with high transcription accuracy and acceptable speed, when compared to MedASR and NVIDIA Canary Qwen. For documentation, presentation and meeting logs, we used Typst, so it made for the perfect PDF generator. The LLM model was selected with testing as well; with LFM2 from LiquidAI, it was faster and a better fit overall compared to Qwen3, Phi-3.5 and Medgemma.

The evaluation of the different technologies selected for the project will be covered in detail in the validation chapter of the report.

Layer	Technology	Purpose
Frontend	Next.js / TypeScript	User/Doctor interface
Backend	C# - ASP.NET	Orchestration & business logic
Speech-to-Text	Faster-Whisper	Audio transcription
LLM Runtime	Ollama + LLM models	Local AI
PDF Generator	Typst	PDF generation
Email	Mailpit	Email sending
Data	PostgreSQL + MongoDB	Un/structured storage
Deployment	Docker Compose	Containerized services & networking

Table 1: Application tech stack

6.3. Frontend

So the users do not have to manage API requests manually, we designed frontend applications, that translate the user's clicks to API calls and present the responses of those requests. We designed two frontends for our project, one for doctors, and one for the patients. Both frontends were designed based on their respective user flows (see Appendix Figure E for the patient flow, and Appendix Figure F for the doctor flow).

We designed two standalone frontends, instead of one with role-based authorization, because in the real world the doctor's frontend would be deployed only locally, within the network of the medical institution, while the patients' frontend would be accessible from everywhere in the world.

It's also important to mention that the frontend for patients is only a proof-of-concept. It was only designed, so that we can present the entire usage workflow, all the way from start to finish.

6.4. Backend architecture

The backend layer consists of two separate ASP.NET Core services written in C#.

6.4.1. Tech stack justification

C# was chosen for its strong static typing, which makes complex domain models easier to reason about and catches errors at compile time rather than at runtime.

ASP.NET Core ships with a built-in dependency injection container, a structured middleware pipeline, and an Auth0 library, covering all core infrastructure needs without additional packages. Python's typical advantage in ML-adjacent work does not apply here, since the LLM services are isolated behind HTTP APIs that the backend simply calls.

6.4.2. Booking backend as proof of concept

The booking backend (Janus) exists primarily to complete the end-to-end patient workflow and to give the consultation backend the patient email address needed to deliver the doctor's note. Core booking and availability functionality is implemented. Certain business logic is intentionally out of scope as the focus of this project is the consultation workflow. The UML diagram in Appendix Figure I illustrates the booking backend's layer structure.

6.4.3. Controller separation

The consultation backend (Heimdall) is divided into four controllers, each responsible for exactly one stage of the clinical workflow. This maps directly onto the use cases defined in the analysis phase:

- **ConsultationController:** Manages consultation record creation and retrieval. This is the entry point for tying an appointment to an active consultation.
- **TranscriptController:** Accepts audio upload, delegates transcription to Echo, and stores the result.
- **SummaryController:** Handles LLM-generated summary creation, editing, and retrieval.
- **PrescriptionController:** LLM draft generation, doctor review and editing, approval, PDF export via Saga, and email delivery via Hermes.

6.4.4. Service and infrastructure layers

Controllers in both backends are kept thin: they parse the incoming HTTP request, delegate to a service interface, and return a response. All business logic lives in the service layer, injected through interfaces. This decoupling means each service is independently testable without needing an HTTP context.

In Heimdall, external dependencies are encapsulated in a dedicated Infrastructure layer. Each external service is wrapped behind a typed interface and registered as a named `HttpClient` through ASP.NET's dependency injection. The rest of the codebase depends only on the interface, not the implementation. Replacing the LLM provider, for example, only requires a new ILLM implementation with no changes to any controller or service.

6.4.5. Data Transfer Objects

Both backends use request and response DTOs to decouple the API contract from the internal data model. Request DTOs define exactly what the caller must send. Response DTOs define what gets returned.

This prevents internal fields, generated identifiers, and database-specific properties from leaking into the API surface, and allows the internal model to change without breaking the external contract.

6.4.6. Error handling

Errors are handled at the controller level through typed exception mapping: `KeyNotFoundException` returns 404 Not Found, while `InvalidOperationException` and `TimeoutException` return 400 Bad Request. This keeps the mapping between domain errors and HTTP status codes explicit and readable without requiring global error middleware.

6.4.7. Authentication and authorization

Both backends use Auth0 for authentication, integrated via JWT Bearer tokens. A JWT is a signed, self-contained token that encodes claims about the user, such as their identity and role. It is issued by Auth0 after a successful login. The backend does not need to call Auth0 on every request, it simply verifies the token's signature using Auth0's public key and checks that it has not expired.

In ASP.NET, authentication is wired into the middleware pipeline in `Program.cs` via `AddAuth0ApiAuthentication`. The `[Authorize]` attribute is then applied at the controller class level, meaning every endpoint requires a valid token by default. The one exception is `GET /api/Availability/doctors`, which carries `[AllowAnonymous]`.

On the frontend, Auth0 handles the login redirect. A server-side `/api/access-token` route is exposed by Next.js so the browser-side client can retrieve its current JWT, which is then forwarded as the Bearer header with every backend API call.

6.4.8. API endpoints

The tables below list all HTTP endpoints exposed by each backend.

Method	Endpoint	Description	Auth
POST	/api/Appointment	Create a new appointment and send a confirmation email	Required
GET	/api/Appointment	Retrieve all appointments for the authenticated user	Required
GET	/api/Availability/doctors	List all doctors and their available time slots	Public

Table 2: Janus (Booking Backend) API endpoints

Method	Endpoint	Description	Auth
POST	/api/Consultation/StartConsultation	Create a consultation record linked to an appointment	Required
GET	/api/Consultation/GetConsultation	Retrieve consultation metadata by ID	Required
GET	/api/Consultation/GetDoctorAppointments	List all appointments for the authenticated doctor	Required
POST	/api/Transcript/GenerateTranscript	Upload audio and trigger transcription via Echo	Required
GET	/api/Transcript/GetTranscript	Retrieve the stored transcript	Required
POST	/api/Summary/GenerateSummary	Generate a clinical summary via the LLM	Required
GET	/api/Summary/GetSummary	Retrieve the stored summary	Required
PUT	/api/Summary/EditSummary	Update the summary with doctor edits	Required
POST	/api/Prescription/GeneratePrescription	Generate a prescription draft via the LLM	Required
GET	/api/Prescription/GetPrescription	Retrieve the stored prescription	Required
PUT	/api/Prescription/EditPrescription	Update the prescription with doctor edits	Required
POST	/api/Prescription/ApprovePrescription	Approve, generate PDF via Saga, and email to patient	Required

Table 3: Heimdall (Consultation Backend) API endpoints

6.5. Database design

As previously mentioned, we have two different kinds of databases. One of them is the relational database, PostgreSQL in our case, for the structured booking data. The

second one is a non-relational database, MongoDB in our case, for patient health data documents.

We decided to go with this design, because PostgreSQL can handle transactional records where consistency, relations and constraints are important, while MongoDB is a better fit for generated documents such as transcripts and summaries, all the while being faster for document-oriented queries and more flexible than its relational counterpart.

During the design phase, the most important aspect for the databases were ACID properties, speed and ease-of-use with the built-in object-relational mapping.

6.5.1. Relational database

The relational model is designed to reflect the booking workflow:

- doctors and patients are the core tables, they are also connected with the Auth0 component
- appointments handle the metadata such as time, status and date. It's also a junction point for both doctors and patients modelling a many-to-many relationship.
- works_on was designed as an extension to the booking. It would only start working after deployment, as it is time consuming to mock and test during development.

6.5.2. Non-relational database

MongoDB stores the documents that we create via the LLM. These are large and semi-structured, allowing us to keep AI outputs from the relational database, while still being able to easily query them by appointment ids. The collections are consultations, raw_transcripts, summaries, and doctor_notes. These are responsible for transcripts, summaries and the generated doctor's note.

For data persistence, Docker mounts volumes, so data is saved even after a restart. This allows us to look back at past records and possibly fine-tune the agents prompt to match our standards.

Appendix Figure J shows our entity relationship diagram.

7. Implementation

7.1. Naming conventions

To make the naming conventions clear in the system architecture, we assigned each component a distinct name to make service boundaries easier to identify.

Name	Function
Echo	Speech-to-text
Heimdall	Consultation Backend
Hermes	Email
Mneme	Database
Odin	LLM
Saga	PDF Generation
Janus	Booking Backend
Eir	Consultation Frontend
Iris	Booking Frontend

7.2. Containerization

Docker and Docker Compose were utilized to containerize each component to keep the environment consistent. A single `docker-compose.yml` at the repository root is responsible for the orchestration of the entire system.

7.2.1. Startup Order & Healthchecks

Startup order is enforced through Docker's `depends_on` conditions. Four services expose healthcheck endpoints that Docker polls before marking them ready.

- **PostgreSQL's** `pg_isready` confirms the database accepts connections. (see Appendix Figure K)
- **MongoDB's** `mongosh` makes sure that the document store is responsive.
- **Echo's** Python `urllib` request ensures that the STT service's model has loaded.
- **Saga's** `curl` request confirms the PDF service is ready.
- **Heimdall** and **Janus** declare `condition: service_healthy` for these four dependencies, ensuring neither backend starts before its data stores and other services have become fully available.
- **Odin** and **Hermes** use `condition: service_started` since neither service exposes a meaningful ready signal.

7.2.2. Environment Variables

Database credentials are read from an `.env` file. Service-to-service URLs are injected as environment variables at the container level, using Docker Compose's internal DNS to resolve service names (e.g., `http://odin:11434`).

7.2.3. Persistent Volumes

Four named volumes ensure that container restarts do not delete data:

- pgdata - PostgreSQL database files. (see Appendix Figure K)
- mongodata - MongoDB database files.
- ollama_data - Downloaded LLM model weights, so the model is not re-pulled on every restart.
- hermes_data - Mailpit's email database.

All services are configured with `restart: unless-stopped`, so the stack recovers automatically from individual container failures with no need of manual intervention.

7.3. Service-by-Service Implementation

7.3.1. Speech-to-Text Service (Echo)

A single processing endpoint `POST /process` accepts an audio file in the request and returns a JSON response. The uploaded audio is written to a temporary file and is deleted immediately after transcription. A `GET /ping` endpoint serves as the health check that confirms the service is ready.

7.3.2. LLM Service (Odin)

On container startup, `entrypoint.sh` checks whether the required model `sam860/LFM2:2.6b` is present and pulls it if missing. Since `/root/.ollama` is mounted to the `ollama_data` volume, the model is persisted and only needs to be pulled once. Odin is called from Heimdall via the `POST /api/chat` endpoint, which accepts a JSON request body containing the model and input messages and returns the generated response.

7.3.3. PDF Generation Service (Saga)

`POST /generate` accepts a JSON body with data of the doctor, patient, diagnosis, description, and prescription. With that information, a JSON file buffer is created. Afterwards, the data is passed against the note template and a PDF file is created. The service reads the file into memory, sends it back as an HTTP response, and deletes the file so it doesn't persist. When Heimdall calls the service, the file gets attached to the email sent via Hermes.

7.3.4. Email Service (Hermes)

Hermes is the service that sends the generated doctor's note to the patients as well as booking confirmations. It uses the `axllent/mailpit` Docker image. `POST /api/v1/send` requests a PDF from the Typst service, then sends it to Hermes as an SMTP message with the PDF attached.

7.3.5. Booking Backend (Janus)

This backend has two controllers:

- AppointmentController

- POST /api/appointment creates a booking and triggers a confirmation email via Hermes.
- GET /api/appointment returns the user's appointments (see Appendix Figure L).
- AvailabilityController
 - GET /api/availability/doctors returns all doctors and their available time slots.

AppointmentService calls IEmail after saving the appointment and updates EmailSentAt once the email is sent.

7.3.6. Consultation Backend (Heimdall)

The consultation backend has four controllers:

- ConsultationController
 - POST /api/consultation/StartConsultation creates a consultation record in MongoDB tied to an existing appointment.
- TranscriptController
 - POST /api/transcript/GenerateTranscript accepts the audio file, forwards it to Echo and stores the result in raw_transcripts.
- SummaryController
 - POST /api/summary/GenerateSummary sends the transcript to Odin and stores the output in summaries.
 - PUT /api/summary/EditSummary allows the doctor to edit the generated summary.
- PrescriptionController
 - POST /api/prescription/GeneratePrescription sends the summary to Odin again and stores the prescription and suggestions in summaries. (see Appendix Figure M)
 - PUT /api/prescription/EditPrescription allows the doctor to rewrite the prescription.
 - POST /api/prescription/ApprovePrescription triggers Saga for PDF generation then Hermes to email it to the patient.

7.3.7. Booking Frontend (Iris)

The first page contains the authentication prompt. (iris/app/page.tsx) Iris' booking page displays the patient's existing bookings and allows creating a new one through a form with doctor selection and a date/time picker. (iris/app/booking/page.tsx) (see Appendix Figure N and Appendix Figure O)

7.3.8. Consultation Frontend (Eir)

There are four pages in this frontend:

Firstly, the dashboard shows the authentication prompt and all appointments for the day, before and after logging in respectively. (eir/app/page.tsx) (see Appendix Figure P)

Secondly, the doctor can begin the consultation. (`eir/app/appointment/[id]/page.tsx`) (see Appendix Figure Q)

Thirdly, the generated transcript is shown and ready for review. (`appointment/[id]/transcript/page.tsx`)

Lastly, the prescription is up for editing and approval. (`appointment/[id]/note/page.tsx`)

7.4. Inter-Service Implementation

Each external service is registered in `Program.cs` as a typed `HttpClient` (see Appendix Figure R).

Each service has a defined interface (`ILLM`, `IPdf`, `ISpeechToText`, `IEmail`) implemented in the `Infrastructure/` layer of the consultation backend. This acts as an abstraction for a translation layer between the business logic and the external service APIs.

7.5. Authentication & Authorization

Both backends use Auth0 JWT Bearer authentication, configured in `Program.cs` via `AddAuth0ApiAuthentication` with domain and audience read from `appsettings.json`.

Each backend has its own Auth0 tenant and audience — Heimdall expects <https://consultation-api> and Janus expects <https://booking-api>.

`[Authorize]` is applied at the controller class level in both backends, so every endpoint requires a valid token by default.

Both frontends use Auth0 to handle the login flow. Both frontends expose an `/api/access-token` route that the client calls to retrieve the token, which is then forwarded to Heimdall with each request.

8. Validation

8.1. Testing

The project employs a carefully selected technology stack for quality assurance and validation across multiple components.

8.1.1. Consultation Backend - ASP.NET

As both backends are built using ASP.NET, we utilized xUnit as the primary testing framework, which is essential for testing asynchronous operations in the backend. Moq provides flexible mocking capabilities, allowing isolation of external dependencies (MongoDB, HTTP services, email services) during testing. This combination enables comprehensive service-level testing without requiring live instances of MongoDB or third-party APIs.

Consultation Service Tests

What is tested:

- The ability to retrieve consultation data and create new consultations.
- The ability to preserve all relevant metadata when creating a consultation record in MongoDB and assign a unique consultation ID.
- The ability to retrieve consultation data and map it to the API response format, ensuring data integrity across relational and non-relational storage layers.

Transcript Service Tests

What is tested:

- The ability to upload audio recordings to the speech-to-text service and retrieve accurate transcriptions, ensuring that the transcription process is correctly integrated and functional.
- Ensuring stored transcriptions are accurately retrieved.

Summary Service Tests

What is tested:

- The test mocks the LLM service to verify that the service correctly passes the transcription to the language model.
- Confirming that the service retrieves the most recent summary version as multiple versions might exist.

Prescription Service Tests

What is tested:

- Testing if the service correctly orchestrates between the summary and LLM service and if the output is correctly structured and tracked.
- Ensuring the latest approved prescription is retrieved.

Mocking Strategy

External dependencies are mocked to isolate business logic:

- **ILLM Service:** Mocked to return controlled and realistic outputs such as structured medical recommendations.
- **ISpeechToText Service:** Mocked to simulate transcription without requiring actual audio processing.
- **IPdf and IEmail Services:** Mocked to avoid side effects during testing.
- **MongoDB Collections:** Mocked using Moq to capture inserted documents and verify persistence without requiring a live database.

The use of callbacks captures documents during insertion, allowing tests to verify both that the data was stored and that the content is correct.

8.1.2. Booking Backend - ASP.NET

The Booking Backend utilizes the same testing framework as the Consultation Backend: xUnit with Moq. This consistency across backends enables shared testing patterns and allows both services to be validated using identical mocking and isolation strategies.

The testing approach focuses on service-level unit tests that validate the business logic of core operations:

Appointment Service Tests

What is tested:

- The ability to save appointments to the database with all metadata as well as generate a unique appointment ID.
- The ability to persist appointment data to the relational database.
- The automatic handling of email sending, including setting the `EmailSentAt` timestamp when emails are successfully sent to patients.
- Confirming that the email contains relevant details such as the assigned doctor's name and appointment time.
- The ability to retrieve all stored appointments and map them correctly to Data Transfer Objects for API responses.

Availability Service Tests

What is tested:

- The ability to retrieve all doctors in the system along with their availability schedules.
- The correct retrieval of all doctors when multiple doctors are present in the system.

Mocking Strategy

External dependencies are mocked to isolate business logic:

- **IEmail Service:** Mocked to simulate email sending without actually dispatching emails, allowing tests to verify the email request content and success/failure handling without external dependencies.

8.1.3. Test Coverage and Results for Backends

Code coverage was measured for both backend services using Coverlet and ReportGenerator, following the .NET testing coverage guidelines.

The overall line coverage for the Consultation Backend was 19.1% and for the Booking Backend it was 9.8%. These figures are deceiving and low due to the infrastructure of both backends. There are many services used, such as Speech-to-Text or Email, that depend on external services and are not suitable for unit testing. Migrations and auto-generated code also significantly contribute to the uncovered line count.

When looking at core logic, we get a much more representative result. In the Consultation Backend the service layer achieves between 48.6% and 73.3%, with the data models reaching 81.8% to 100% (see Appendix Figure S). In the Booking Backend, the appointment and availability services both achieve 100% (see Appendix Figure T).

8.1.4. LLM Service - Liquid AI¹⁴

LLM model selection involved comparative benchmarking using two evaluation approaches. The Summary and Suggestion quality assessment employed ROUGE-1 and BERTScore metrics to measure output accuracy and semantic understanding while the MedQA benchmark assessed medical knowledge accuracy on a standardized dataset of 50 medical multiple-choice questions. This dual-metric approach ensured the selected model (Liquid AI LFM2) balanced both clinical relevance and real-time performance requirements.

Testing Strategy Rather than unit tests, this section uses benchmark metrics to assess output quality and operational performance characteristics. Two models were evaluated: Liquid AI (LFM2 2.6B parameters) and Gemma 4 (4B parameters).

The testing approach focuses on two critical dimensions:

1. Summary and suggestion quality for clinical accuracy.
2. Medical knowledge assessment against benchmark datasets.

Summary & Suggestion Quality Assessment

What is tested:

- The ability to generate accurate clinical summaries from consultation transcripts using ROUGE-1 metric, which measures individual word matches and validates that key clinical terms are preserved.
- The model's contextual understanding of medical concepts using BERTScore F1 metric, which evaluates semantic similarity and ensures the generated summaries maintain clinical meaning.
- Latency and token throughput to ensure the model operates efficiently for real-time clinical use, with tests run on transcripts ranging from 1 to 14 minutes.

¹⁴docs/research/LLM-testing.typ

Medical Knowledge Assessment (MedQA Benchmark)

What is tested:

- The model's ability to correctly answer medical multiple-choice questions from the MedQA benchmark dataset, measuring medical knowledge accuracy.
- Overall performance on 50 representative questions from the 1000+ question medical knowledge base.
- Latency and token throughput during medical question answering to assess real-time diagnostic support feasibility.

Summary Quality Results: Liquid AI demonstrated superior performance for clinical summary generation:

- ROUGE-1 average: 0.562 vs. Gemma 4 at 0.487
- BERTScore F1 average: 0.385 vs. Gemma 4 at 0.341
- Average latency: 110.40s vs. Gemma 4 at 267.85s
- Throughput: 13.38 tokens/sec vs. Gemma 4 at 8.16 tokens/sec

Medical Knowledge Results: Gemma 4 demonstrated superior accuracy on medical knowledge questions:

- Accuracy: 74.0% vs Liquid AI at 48.0%
- However, latency was significantly higher at 80.97s average vs 15.38s for Liquid AI

8.1.5. Speech-to-text Service - Faster-Whisper¹⁵

The Speech-to-Text service runs Faster-Whisper, an optimized implementation of OpenAI's Whisper model for speech recognition. Performance validation employs custom benchmark tooling that measures throughput, latency, and failure rates under varying concurrent loads. This benchmarking approach was used to validate that the service meets our requirements, where transcription must be completed within acceptable timeframes for doctor-patient consultations.

Testing Strategy

The Speech-to-Text Service validation employs stress testing and benchmark analysis to evaluate performance and reliability under varying loads.

The testing approach employs four sequential test phases to establish performance characteristics:

1. Baseline testing at low concurrency to establish normal operation.
2. Ramp stress testing with gradually increasing concurrency.
3. Peak stress testing at maximum expected load.
4. Recovery testing to verify the service restores to baseline performance.

Test Methodology

What is tested:

¹⁵docs/research/STT-test.typ

- The service's ability to process audio transcription requests at low concurrency (1-2 concurrent users) to establish baseline throughput and latency metrics.
- Performance degradation under gradually increasing concurrent load (1-16 concurrent requests) to identify where performance begins to degrade.
- Behavior under peak stress conditions (20-24 concurrent requests) to assess maximum capacity and failure rates.
- Recovery and stability after stress testing to ensure no permanent degradation from heavy load scenarios.

The tests use the same audio file across all phases. The file is a mash-up of audio clips from Mozilla Common Voice dataset. Concurrency levels represent simultaneous transcription requests. Key metrics measured include throughput (requests/second), mean latency, 95th percentile latency (p95), and failure rate.

Baseline Performance (Concurrency 1-2):

- Concurrency 1: 0.401 requests/sec, 2.49s mean latency, 0% failure rate
- Concurrency 2: 0.398 requests/sec, 4.94s mean latency, 0% failure rate

Ramp Stress Results (Concurrency 1-16):

- Throughput remained stable at ~0.420 requests/sec across all concurrency levels
- Mean latency increased proportionally with concurrency:
 - Concurrency 4: 9.38s latency
 - Concurrency 8: 18.38s latency
 - Concurrency 12: 26.93s latency
 - Concurrency 16: 35.20s latency
- Zero failures at all concurrency levels

Peak Stress Results (Concurrency 20-24):

- Concurrency 20: 0.420 requests/sec, 43.85s mean latency, 0% failure rate
- Concurrency 24: 0.420 requests/sec, 51.64s mean latency, 0% failure rate

Recovery Test (Return to Concurrency 1):

- Mean latency: 2.41s (comparable to baseline 2.49s)
- Throughput: 0.415 requests/sec (baseline was 0.401)
- Failure rate: 0% (unchanged)

Performance Analysis

- **Stable Throughput:** The service maintains approximately 0.42 requests/sec regardless of concurrency level, indicating that request processing is sequential and request queuing, rather than parallelization, limits throughput.
- **Linear Latency Scaling:** Mean latency increases linearly with concurrency, which is expected when requests are queued. With stable throughput and proportional latency, the system remains predictable and reliable.
- **Zero Failure Rate:** No requests failed during any test phase, including peak stress conditions, demonstrating robust error handling and no resource exhaustion at the tested concurrency levels.

- **Full Recovery:** The recovery test shows the service returns to baseline performance, indicating no permanent degradation or resource leaks from sustained stress testing.

The service represents a strong baseline suitable for stable, low-concurrency operation typical of a single outpatient department.

8.1.6. Manual Testing

Test Case ID	Test Scenario	Test Steps	Actual Result	Status
TC-001	Login via Iris/Eir	1. Navigate to the Iris/Eir login page. 2. Enter valid patient credentials. 3. Submit the login form.	User is authenticated and redirected to the booking/consultation dashboard	Pass
TC-002	Start a consultation	1. Log in as a doctor. 2. Select relevant appointment. 3. Press start consultation. 4. Talk with patient. 5. Edit summary.	The full doctor's note is generated with symptoms, diagnosis, description and advice or prescription and is stored in the non-relational database.	Pass
TC-003	Book an appointment	1. Log in as a patient. 2. Select a doctor and an available time slot. 3. Confirm the booking.	Booking is created, appears on the dashboard and is visible in the doctor's consultation view.	Pass
TC-004	Register with Iris/Eir	1. Press sign up. 2. Enter email and password. 3. Send the form. 4. Log in.	User is created in the local relational database and the Auth0 database.	Pass

Table 4: Manual test cases

¹⁶docs/research/authentication.typ

8.2. Supporting Technologies

8.2.1. Auth0 for Authentication¹⁶

Authentication is provided by Auth0, a third-party identity platform with medical SSO compliance. This choice eliminates the need for custom authentication testing, with Auth0's certified compliance supporting HIPAA and other healthcare standards.

8.2.2. Mailpit for Email Testing¹⁷

Email functionality uses Mailpit for development and testing environments, allowing complete control over email delivery without external dependencies. Mailpit runs locally in Docker, enabling email integration testing without affecting external systems.

8.2.3. Typst for PDF Generation¹⁸

PDF prescriptions are generated using Typst, a text-based document system that allows data-driven PDF generation. Typst's great customizability and data templating capabilities make it suitable for clinical prescription documents with structured, reusable formatting.

8.2.4. Docker and Docker Compose

The entire system is containerized and orchestrated using Docker and Docker Compose, enabling consistent testing environments across development, CI/CD, and deployment stages. Each service runs in isolated containers.

This containerization enables:

- Reproducible testing environments independent of host machine configuration
- Isolated service testing without cross-contamination between tests
- Stress testing of individual services (e.g., STT service under load) in controlled conditions

¹⁷docs/research/email.typ

¹⁸docs/research/pdf.typ

9. Conclusion

9.1. Summary

This semester's project successfully delivered a privacy-preserving OPD (Outpatient Department) management system that meets the main objectives, defined in collaboration with the case owners at the beginning of the semester. Our solution enables doctors to reduce mistakes made during consultations by reducing consultation administrative work, while the system is secure in terms of handling sensitive patient data on the clinic's network. Additionally, a proof of concept for the booking system was developed to demonstrate the end-to-end workflow from patient booking to consultation, while keeping them separate.

Reflecting on the project, the following points were a success:

- CBS design - Independently deployed infrastructure allows for easy testability and modularity during development
- Methodology - Two meetings a week ensures constant accountability and progress
- Auth0 Integration - handled both backends cleanly with minimal friction
- Docker Compose - full system consistency, reproducible across all machines

What could be improved:

- Booking backend/frontend - remains as POC, still lacks business logic validation and development
- Mailpit - remains as a development tool, production would require SMTP service

The system demonstrates the application of the knowledge gained throughout this semester's courses. It demonstrates the design of a component-based system and the use of a locally-hosted large language model.

9.2. Future work

There are many improvements, which can be implemented in the future to enhance the system. In terms of clinical capability, the LLM could be expanded, so it looks at past patient history to propose contraindications or warn about possible allergy-related side effects. We could also make the AI use reinforcement learning, where the doctor can rate the AI's responses to fine-tune the local model over time. Furthermore, AI could help assist the doctor not only with making mistakes and prescriptions, but also in differential diagnosis.

For the frontend part of our project, we did not spend so much time on making it a very accessible platform, as our project is not a booking platform, it's an AI service. However, we could extend it to add past consultation summaries and prescriptions after the fact, appointment reminders via the email service and a preconsultation form that the patient can fill out with symptoms to feed into the LLM context.

Furthermore, the speech-to-text (STT) component currently used in the system is not suited for horizontal scaling, because it cannot handle more than one audio file at the same time. We need to find a solution that is concurrent, either using a job queue or

async transcriptions. This change would allow the parallel processing of the audio recordings, reducing the wait times and making the user experience smoother.

In conclusion, these improvements would greatly enhance user experience and the variety of features offered by our application and would make it one step closer to a real-world deployment.

Appendix

<KEY> <Name of issue>

(eg. OPUS-0 Write a poem)

Description

Give a description of the issue and how you resolved it.

Important changes

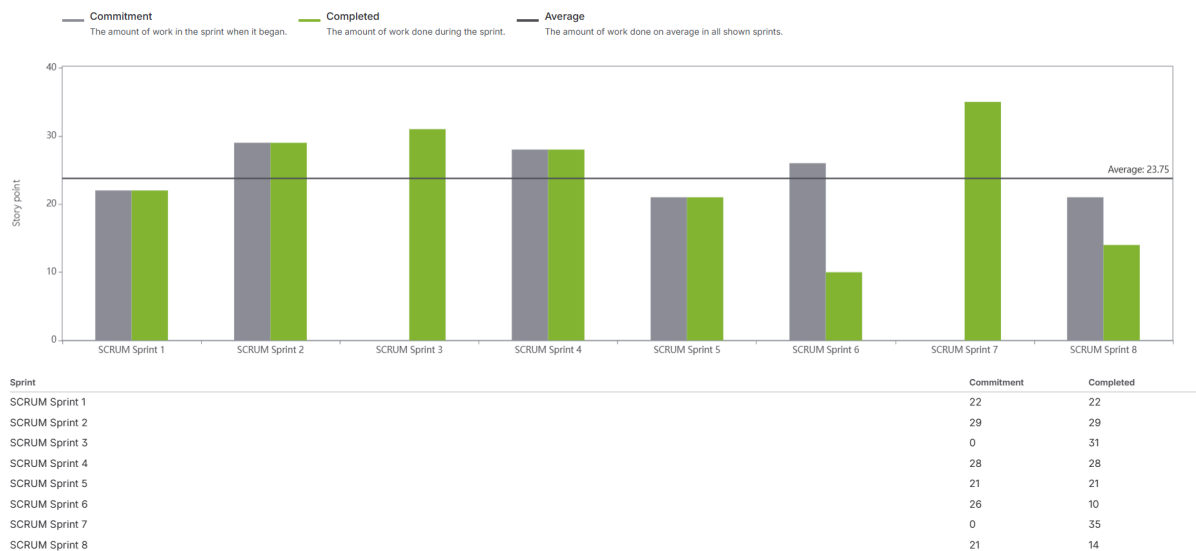
.

Checklist

- ☐ My contribution follows the style guidelines of this project
- ☐ I have performed a self-review of my own contribution
- ☐ I have made corresponding changes in documentation
- ☐ My changes generate no new warnings or errors
- ☐ I have moved my task to **In Review** on Jira

Appendix Figure A: Pull Request Template

Velocity report

[> How to read this report](#)

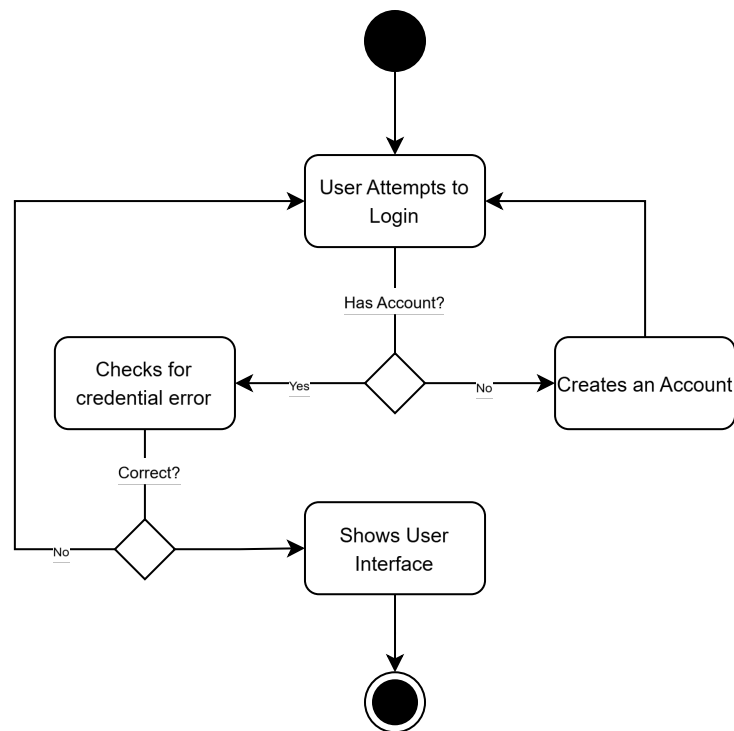
Appendix Figure B: Velocity report chart - Jira

Completed work items [View in All work navigator](#)

Key	Summary	Work type	Epic	Status	Assignee	Story points
OPUS-24	create wireframe	☒ Task		DONE		3
OPUS-28	sign up for auth0	☒ Task		DONE		1
OPUS-26	put all services in a single docker-compose	☒ Task		DONE		5
OPUS-27	create consultation backend	☒ Task		DONE		8
OPUS-29	llm benchmarking	☒ Task		DONE		3
OPUS-30	create booking backend	☒ Task		DONE		8

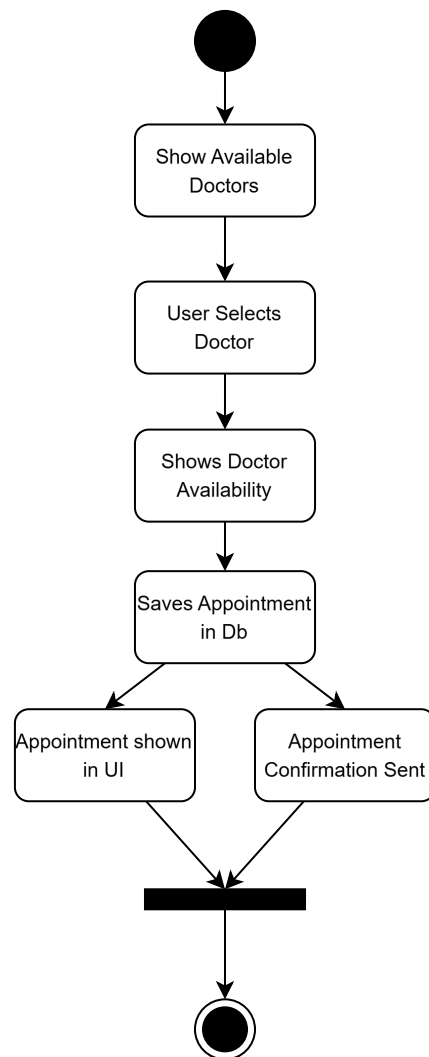
Appendix Figure C: Sprint 4 backlog & Story point - Jira

Login - User Workflow

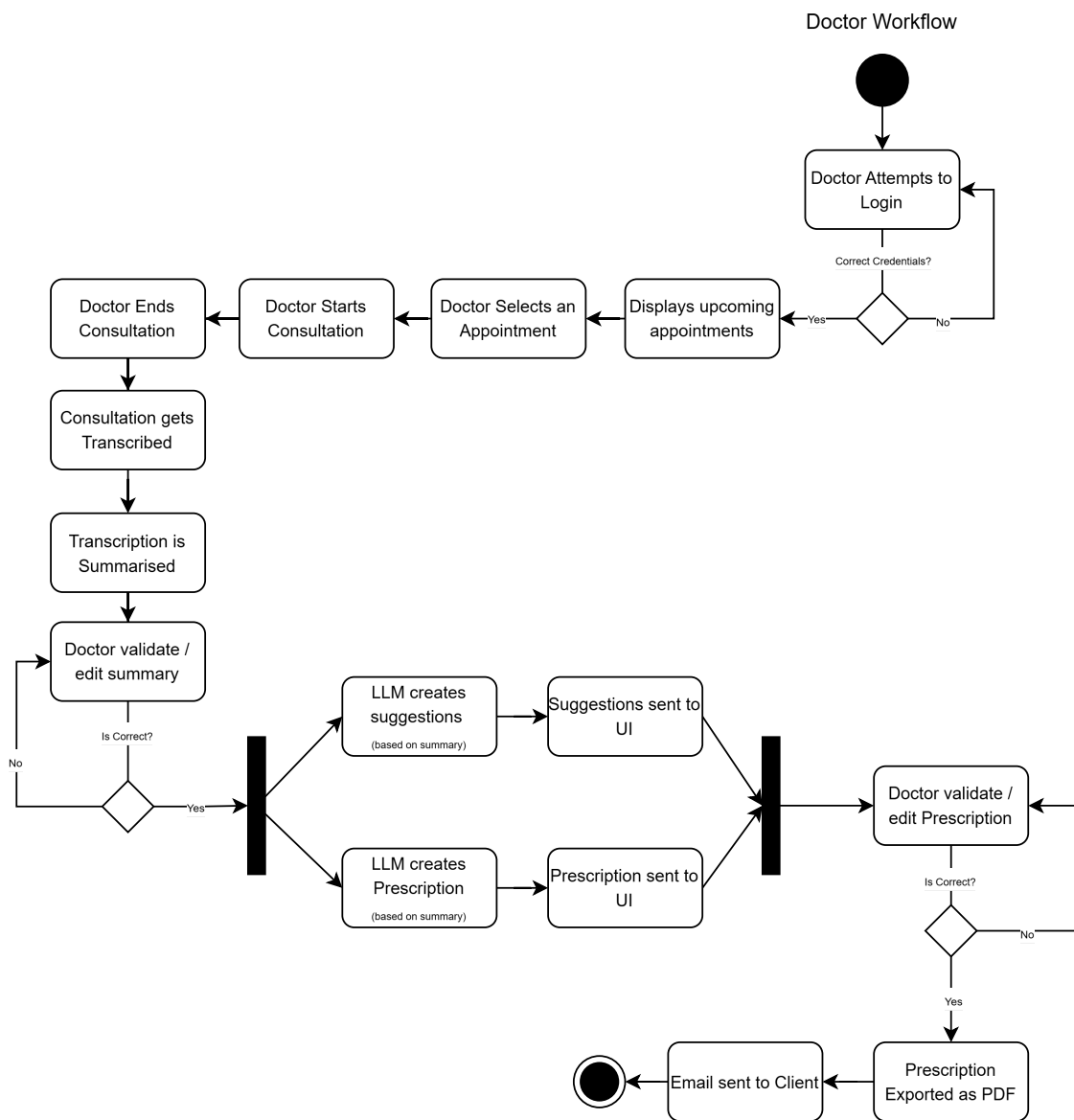


Appendix Figure D: Activity Diagram - Login

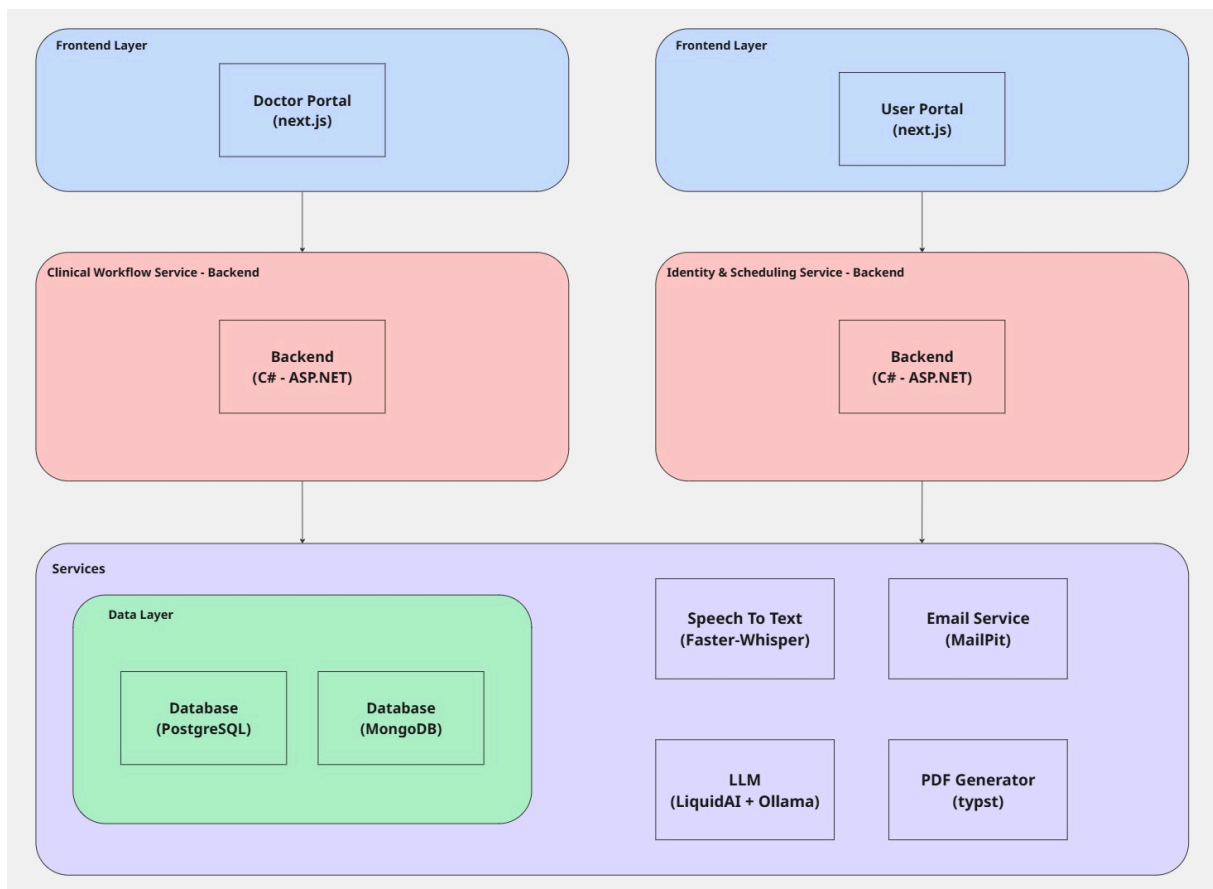
Booking - User Workflow



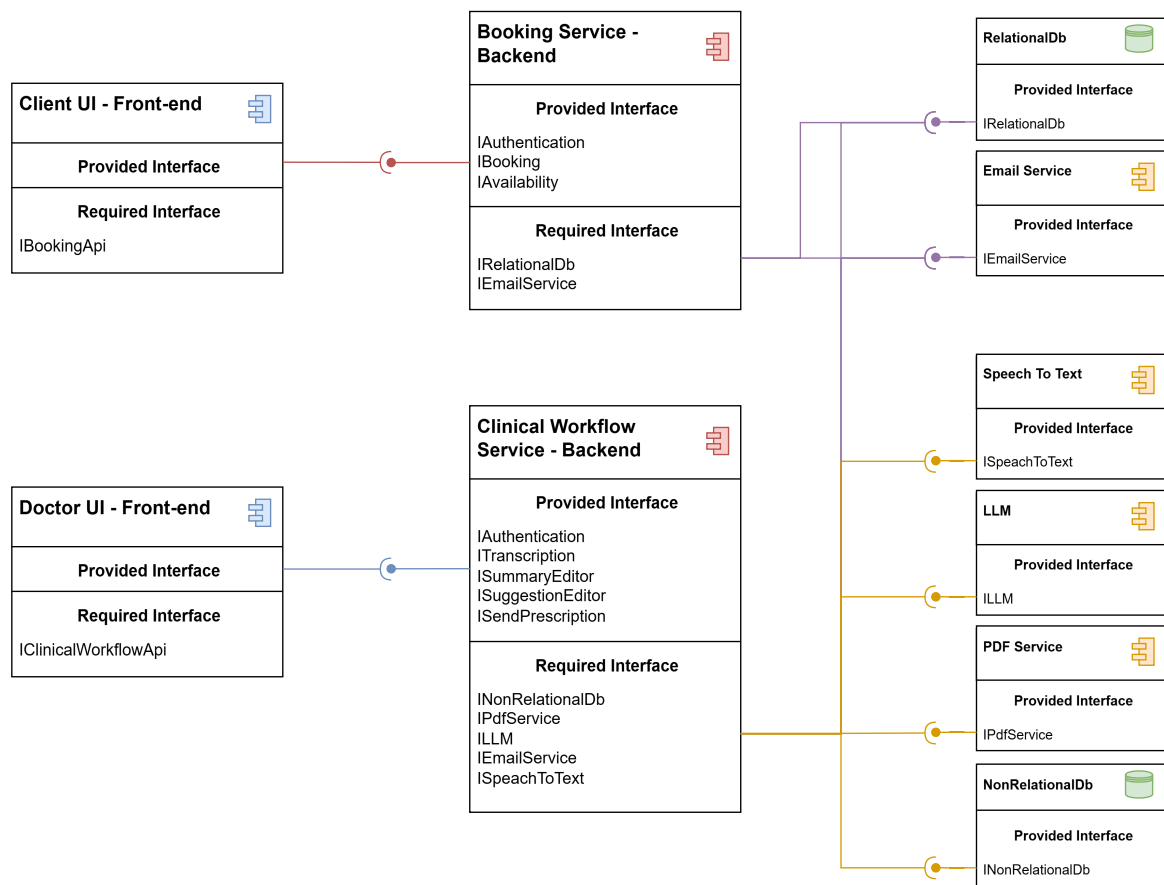
Appendix Figure E: Activity Diagram - Booking



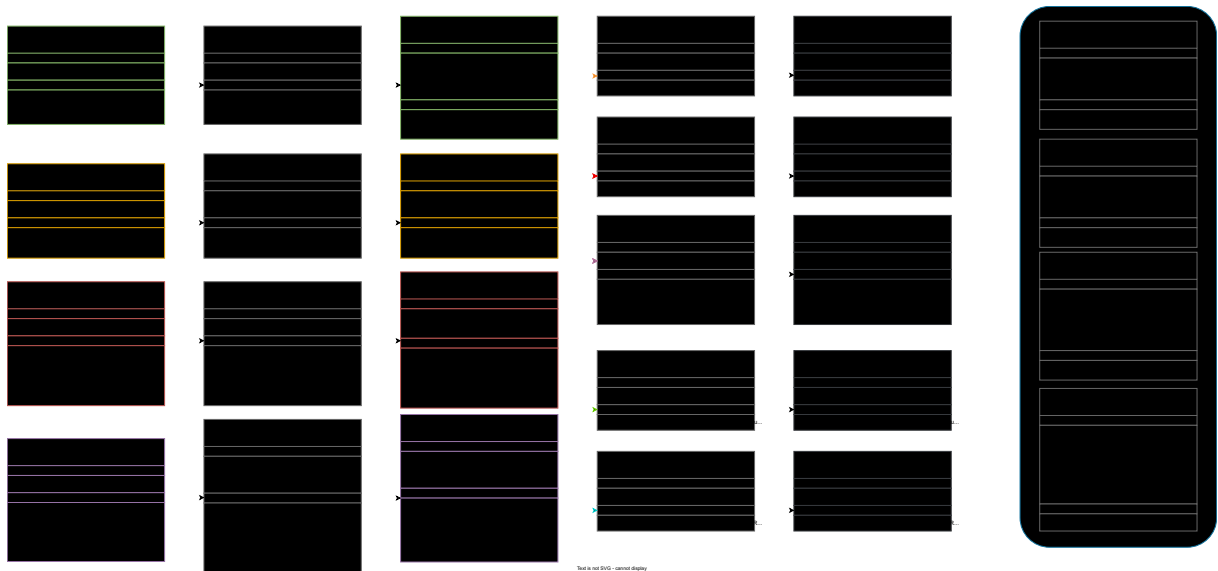
Appendix Figure F: Activity Diagram - Doctor



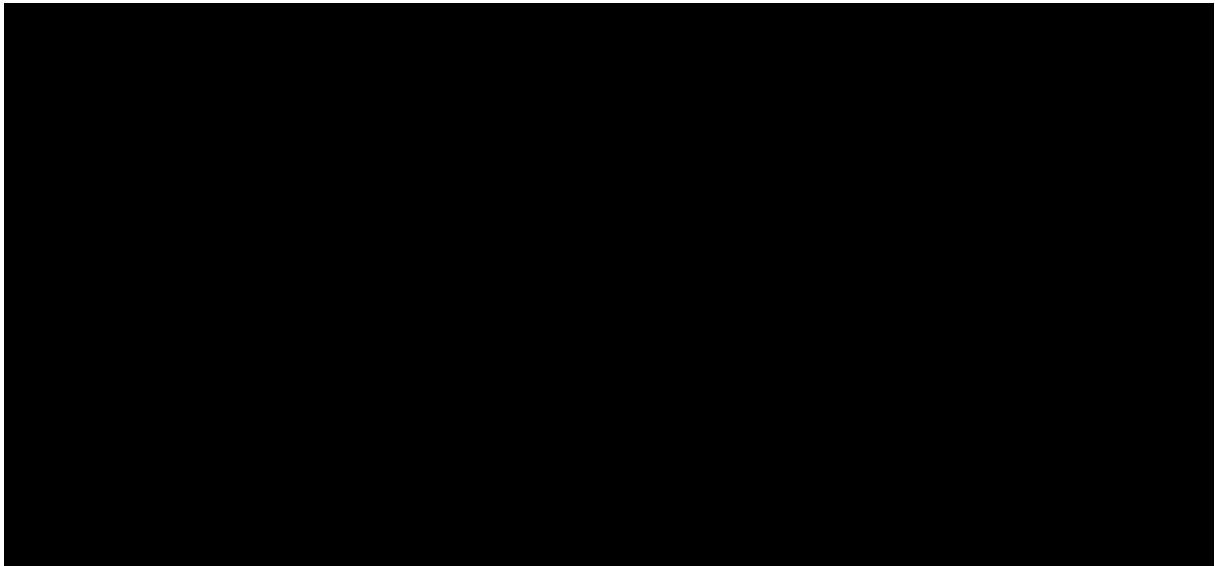
Appendix Figure G: Application Layer Diagram



Appendix Figure H: Component-Based system diagram



Appendix Figure I: UML diagram of the booking backend



Appendix Figure J: Relational database Entity Relationship Diagram

```
▷ Run Service
postgres:
  image: postgres:16
  container_name: postgres
  restart: unless-stopped
  env_file: .env
  environment:
    POSTGRES_DB: ${POSTGRES_DB}
    POSTGRES_USER: ${POSTGRES_USER}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  volumes:
    - pgdata:/var/lib/postgresql/data
  ports:
    - "${POSTGRES_PORT:-5432}:5432"
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 10s
```

Appendix Figure K: PostgreSQL Docker Compose Snippet

```

namespace BookingBackend.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [Authorize]
    1 reference
    public class AppointmentController : ControllerBase
    {
        3 references
        private readonly IAppointment _appointmentService;

        0 references
        public AppointmentController(IAppointment appointmentService)
        {
            _appointmentService = appointmentService;
        }

        [HttpPost]
        0 references
        public async Task<ActionResult<AppointmentDTO>> SaveAppointment([FromBody] CreateAppointmentDTO dto)
        {
            var result = await _appointmentService.SaveAppointmentAsync(dto);
            return CreatedAtAction(nameof(GetAppointments), result);
        }

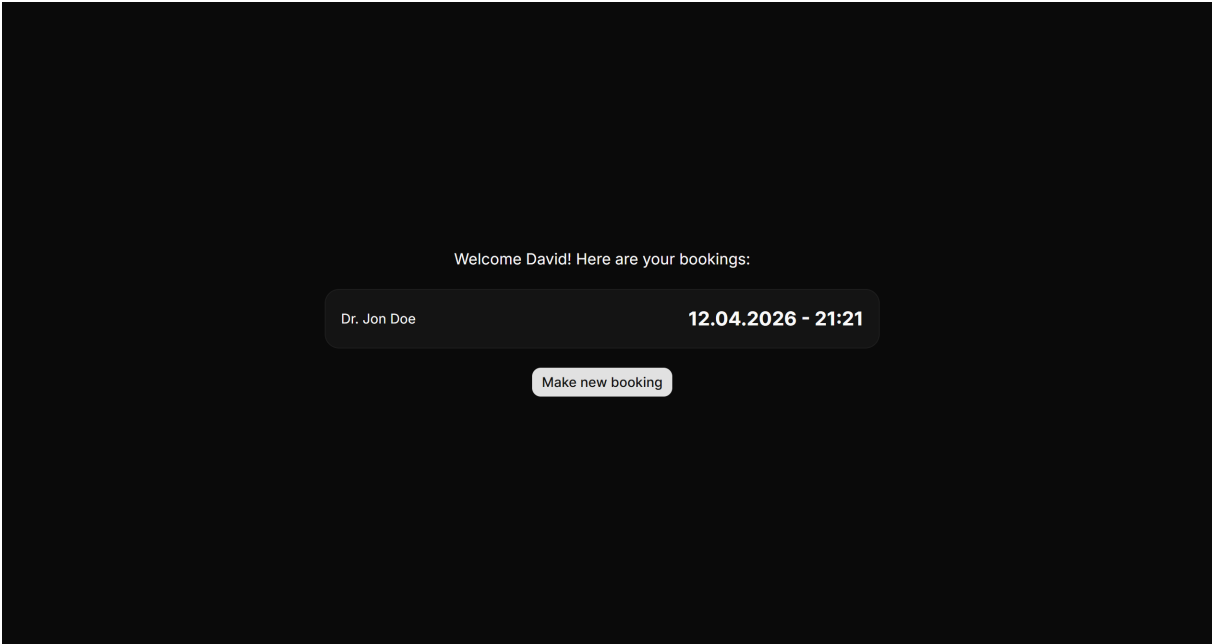
        [HttpGet]
        1 reference
        public async Task<ActionResult<List<AppointmentDTO>>> GetAppointments()
        {
            var appointments = await _appointmentService.GetAppointmentsAsync();
            return Ok(appointments);
        }
    }
}

```

Appendix Figure L: Appointment Controller

```
[HttpPost("GeneratePrescription")]
0 references
public async Task<IActionResult> GeneratePrescription(Guid consultationId)
{
    try
    {
        var note = await _prescriptionService.GeneratePrescription(consultationId);
        return Ok(new { note });
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(ex.Message);
    }
    catch (TimeoutException ex)
    {
        return BadRequest(ex.Message);
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(ex.Message);
    }
}
```

Appendix Figure M: Generate Prescription



Appendix Figure N: Iris - Bookings List

Dear David, add your booking details:

Doctor

Dr. Jane Doe

Date

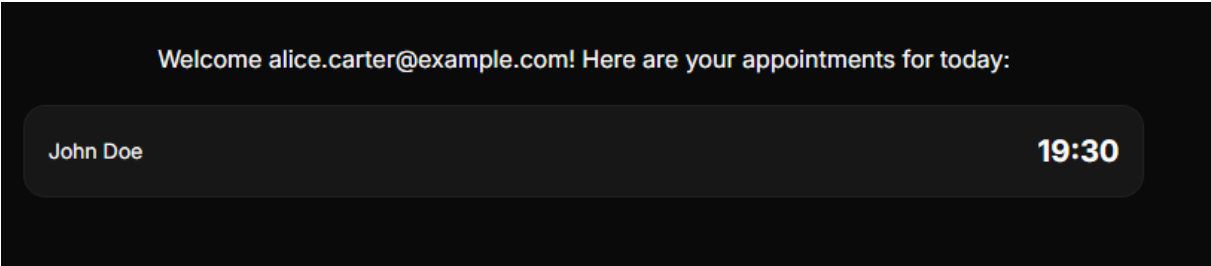
April 30th, 2026

Time

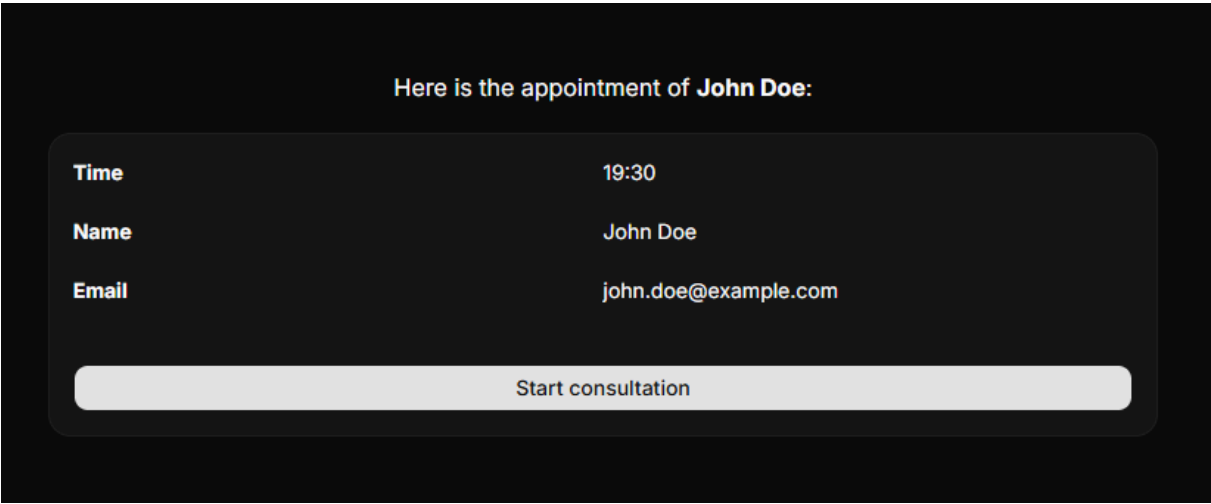
10:30

Make booking

Appendix Figure O: Iris - New Booking Form



Appendix Figure P: Eir - Dashboard



Appendix Figure Q: Eir - Consultation Start

```

builder.Services.AddHttpClient<ILLM, LLM>((sp, client) =>
{
    // this fetches the url from appsetting.json
    var config = sp.GetRequiredService<IConfiguration>();
    var baseUrl = config["Services:LLM:BaseUrl"];

    client.BaseAddress = new Uri(baseUrl!);
});

builder.Services.AddHttpClient<IPdf, Pdf>((sp, client) =>
{
    // this fetches the url from appsetting.json
    var config = sp.GetRequiredService<IConfiguration>();
    var baseUrl = config["Services:Pdf:BaseUrl"];

    client.BaseAddress = new Uri(baseUrl!);
});

builder.Services.AddHttpClient<ISpeechToText, SpeechToText>((sp, client) =>
{
    // this fetches the url from appsetting.json
    var config = sp.GetRequiredService<IConfiguration>();
    var baseUrl = config["Services:SpeechToText:BaseUrl"];

    client.BaseAddress = new Uri(baseUrl!);
});

builder.Services.AddHttpClient<IEmail, Email>((sp, client) =>
{
    // this fetches the url from appsetting.json
    var config = sp.GetRequiredService<IConfiguration>();
    var baseUrl = config["Services:Email:BaseUrl"];

    client.BaseAddress = new Uri(baseUrl!);
});

```

Appendix Figure R: Heimdall Dependency Injection

Name	Covered	Uncovered	Coverable	Total	Percentage	
ConsultationBackend	417	147	564	936	73.9%	
ConsultationBackend.DTOs.BookingRequest	6	0	6	13	100%	
ConsultationBackend.Data.AppDbContext	145	2	147	166	98.6%	
ConsultationBackend.Data.MongoDbContext	8	0	8	23	100%	
ConsultationBackend.Data.MongoDbSettings	2	0	2	12	100%	
ConsultationBackend.Dtos.ConsultationResponse	9	0	9	14	100%	
ConsultationBackend.Models.NonRelational.ConsultationDocument	9	0	9	28	100%	
ConsultationBackend.Models.NonRelational.DoctorNoteDocument	12	1	13	34	92.3%	
ConsultationBackend.Models.NonRelational.RawTranscriptDocument	7	1	8	25	87.5%	
ConsultationBackend.Models.NonRelational.SummaryDocument	9	1	10	32	90%	
ConsultationBackend.Models.Relational.Appointment	9	2	11	22	81.8%	
ConsultationBackend.Models.Relational.Doctor	4	0	4	10	100%	
ConsultationBackend.Models.Relational.Patient	4	0	4	10	100%	
ConsultationBackend.Services.ConsultationService	40	28	68	119	58.8%	
ConsultationBackend.Services.PrescriptionService	71	75	146	220	48.6%	
ConsultationBackend.Services.SummaryService	49	25	74	123	66.2%	
ConsultationBackend.Services.TranscriptService	33	12	45	85	73.3%	

Appendix Figure S: Consulation Test Coverage

Name	Covered	Uncovered	Coverable	Total	Percentage	
BookingBackend	195	0	195	364	100%	
BookingBackend.DTO.AppointmentDTO	7	0	7	21	100%	
BookingBackend.DTO.AvailabilitySlot	3	0	3	16	100%	
BookingBackend.DTO.CreateAppointmentDTO	4	0	4	21	100%	
BookingBackend.DTO.DoctorAvailability	3	0	3	16	100%	
BookingBackend.DTO.EmailGenerateRequest	9	0	9	17	100%	
BookingBackend.Data.BookingDbContext	61	0	61	91	100%	
BookingBackend.Models.Appointment	10	0	10	18	100%	
BookingBackend.Models.Doctor	5	0	5	13	100%	
BookingBackend.Models.Patient	4	0	4	12	100%	
BookingBackend.Models.WorksOn	5	0	5	15	100%	
BookingBackend.Services.Implementations.AppointmentService	62	0	62	87	100%	
BookingBackend.Services.Implementations.AvailabilityService	22	0	22	37	100%	

Appendix Figure T: Booking Test Coverage

Meeting Log

Date and time: 10.02.2026 12:10

Location: SDU Alsion - U110

Attendees: Sean Larsen, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann, Örs Tomaj Jeney, Dávid Borka

- We went through the summaries of each project and picked our two favourites. Afterwards, we dove deeper into those and finally picked **OPD-Vertex**.
- We went through the proposed Group Contract rule-by-rule, discussed and came up with solutions. By the end we got a contract, which everyone agrees to and finds acceptable.
- We appointed a Scrum Master (Igor Zimmermann), who set Jira up for our project.
- We discussed our availability for the Midterm Presentation of the 9th of April and ultimately came up with the fact that we need to move our presentation to a later date.

Appendix Figure U: Meeting Log 1

Meeting Log

Date and time: 16.02.2026 12:45

Location: SDU Alsion - J1.03

Attendees: Sean Larsen, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann, Örs Tomaj Jeney, Dávid Borka

- We signed the Group Contract.
- We contacted the group supervisor.
- We designed the initial component diagram.
- We decided the tech stack.
- We calculated the dependency depth.

Appendix Figure V: Meeting Log 2

Meeting Log

Date and time: 23.02.2026 12:00

Location: SDU Alsion - J1.03

Attendees: Sean Larsen, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann, Örs Tomaj Jeney, Dávid Borka, Riccardo Terrenzi

- We signed the Supervisor Contract.
- We discussed our diagram made in the previous meeting and realised that it's a Application Layer diagram.
- Supervisor recommended that we use a NoSQL database.
- Supervisor recommended that we use LiquidAI or Gwen and run it via Ollama.
- Supervisor proposed we could implement RAG.
- Ákos gave notice that he'll only be available online for the 02.26.
- We broke the project proposal down into tasks, gave story points, and assigned each task.
- We started Sprint 1.
- Igor created the base for the project proposal.
- We went through the user flows.
- We created a component diagram based on the user flow.

Appendix Figure W: Meeting Log 3

Meeting Log

Date and time: 26.02.2026 13:38

Location: SDU Løkken - 104

Attendees: Sean Larsen, *Kristóf Ákos Koltai*, Denis Nedzhibov, Igor Zimmermann, Örs Tomaj Jeney, Dávid Borka

- Team members contributions since last meeting:
 - **Örs:** Task in progress (0PUS-7), content has been written. Only has to copy to project-proposal file.
 - **Dávid:** No contributions yet.
 - **Sean:** Completed task (0PUS-10). Resolved requested changes. PR ready to merge.
 - **Igor:** No contributions yet.
 - **Denis:** No contributions yet.
 - **Ákos:** Has started writing the text for both tasks (0PUS-5, 0PUS-6).
- Updated PR template to include point about moving task to In Review column.

Appendix Figure X: Meeting Log 4

Meeting Log

Date and time: 02.03.2026 12:00

Location: SDU Alsion - J1.18

Attendees: Sean Larsen, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann, Örs Tomaj Jeney, Dávid Borka, Riccardo Terrenzi

- We merged all approved pull requests and closed the first sprint.
- We analyzed a component-based design with our supervisor.
- We went through our Project Proposal with our supervisor.
- Supervisor showed us UCloud.
- We made notes of the changes suggested.
- We split research tasks and put them in Jira.
- We gave story points using Story Point Poker.
- We distributed the issues and started a new sprint.

Appendix Figure Y: Meeting Log 5

Meeting Log

Date and time: 05.03.2026 13:45

Location: SDU Løkken - 104

Attendees: Sean Larsen, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann, Örs Tomaj Jeney, Dávid Borka

- Team members contributions since last meeting:
 - **Örs:** No contributions yet.
 - **Dávid:** Setup LLMs for testing (0PUS-12).
 - **Sean:** Completed task (0PUS-18). Resolved requested changes. PR merged.
 - **Igor:** No contributions yet.
 - **Denis:** No contributions yet. Will not be done with task by Friday.
 - **Ákos:** Has started researching task (0PUS-15).

Appendix Figure Z: Meeting Log 6

Meeting Log

Date and time: 09.03.2026 12:00

Location: SDU Alston - J1.01-C

Attendees: Sean Larsen, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann, Örs Tomaj Jeney, Dávid Borka, Riccardo Terrenzi

- We showed the supervisor our progress and our new diagrams. Based on those he gave us the feedback that we should start implementing from the smallest components, building up to the biggest.
- We went through our flow diagrams, so everyone understands how the application works.
- Everyone presented their research to the rest of the members.
- Final tech stack:
 - Backend: C# ASP.NET
 - Speed-to-text: Faster-Whisper
 - Authentication: Auth0
 - Email: MailPit
 - LLM: LiquidAI
 - PDF Generation: Typst
 - UI: Next.js
- We split service development into tasks, gave story points and assigned tasks.

Appendix Figure AA: Meeting Log 7

Meeting Log

Date and time: 12.03.2026 13:25

Location: SDU Løkken - 104

Attendees: Sean Larsen, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann, Örs Tomaj Jeney, Dávid Borka

- Team members contributions since last meeting:
 - **Örs:** No contributions yet.
 - **Dávid:** Only has last section remaining (0PUS-20).
 - **Sean:** Completed task (0PUS-25). Waiting for review.
 - **Igor:** Has begun development (0PUS-19).
 - **Denis:** Has experimented, only has to document (0PUS-21).
 - **Ákos:** Has started researching task (0PUS-22). Will not open request by Friday.

Appendix Figure AB: Meeting Log 8

Meeting Log

Date and time: 16.03.2026 12:00

Location: SDU Alsion - J1.07

Attendees: Sean Larsen, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann, Örs Tomaj Jeney, Dávid Borka, Riccardo Terrenzi

- We showed the supervisor our progress. Based on those he gave us feedback on what we should do:
 - Stress test / Implementation test
 - Benchmark testing - LLM & Speech-to-text
 - Fault tolerance test
 - Unit testing
- We discussed remaining work to be done on OPUS-23.
- We figured out tasks for the next sprint.
- We gave story point values to tasks and assigned people.
- The scrum master created the missing folders and READMEs.

Appendix Figure AC: Meeting Log 9

Meeting Log

Date and time: 19.03.2026 13:40

Location: SDU Løkken - 104

Attendees: Sean Larsen, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann,
Örs Tomaj Jeney, Dávid Borka

- Team members contributions since last meeting:
 - **Örs:** No contributions yet.
 - **Dávid:** No contributions yet.
 - **Sean:** Working heavily on his task (0PUS-27).
 - **Igor:** Has finished his task (0PUS-24). All approved.
 - **Denis:** Has opened a pull request for his task (0PUS-26).
 - **Ákos:** Has started working on his task (0PUS-29).

Appendix Figure AD: Meeting Log 10

Meeting Log

Date and time: 23.03.2026 12:00

Location: SDU Alsion - J1.07

Attendees: Sean Larsen, *Kristóf Ákos Koltai*, Denis Nedzhibov, Igor Zimmermann, ~~Örs Tomaj Jeney~~, Dávid Borka, Riccardo Terrenzi

Current status of the project: we have currently created the two backends, and set up all the infrastructure. Both backends expose the relevant endpoints and have all PDF, email, LLM and Speech-to-text services connected and functioning.

- We resolved merge conflicts.
- We showed our supervisor our progress.
- He gave us feedback on what we could do for the next Sprint.
- We closed the sprint and came up with new tasks for the next one.
- We decided on Story Point values using Story Point Poker.
- We started the new sprint.

Appendix Figure AE: Meeting Log 11

Meeting Log

Date and time: 26.03.2026 14:30

Location: SDU Løkken - 104

Attendees: Sean Larsen, *Kristóf Ákos Koltai*, Denis Nedzhibov, Igor Zimmermann, *Örs Tomaj Jeney, Dávid Borka*

- Team members contributions since last meeting:
 - **Örs:** No contributions yet.
 - **Dávid:** No contributions yet.
 - **Sean:** Finished his taks (0PUS-36). Awaiting reviews.
 - **Igor:** No contributions yet.
 - **Denis:** No contributions yet.
 - **Ákos:** No contributions yet.

Appendix Figure AF: Meeting Log 12

Meeting Log

Date and time: 15.04.2026 13:00

Location: SDU Alsion - J1.07

Attendees: Sean Larsen, *Kristóf Ákos Koltai*, Denis Nedzhibov, Igor Zimmermann, Örs Tomaj Jeney, Dávid Borka

- We resolved merge conflicts and merged everything afterwards.
- We figured out new tasks and distributed them.
- We gave story point values using Story Point Poker.
- We started the new sprint.

Appendix Figure AG: Meeting Log 13

Meeting Log

Date and time: 20.04.2026 12:00

Location: SDU Alsion - J1.07

Attendees: Sean Larsen, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann, ~~Örs Tomaj Jeney~~, *Dávid Borka*, Riccardo Terrenzi

- We discussed issues between our front-ends and back-ends.
- We discussed what needs to be included in the report, and what materials we can reuse for each one of them.
- We distributed the tasks.
- Supervisor gave us feedback:
 - Include unit testing coverage.
 - Make consultation UI have more functionality.
 - Use Bootstrap CI for LLM testing.
- We played Party Poker to determine story point values.

Appendix Figure AH: Meeting Log 14

Meeting Log

Date and time: 27.04.2026 12:00

Location: SDU Alsion - Software Room

Attendees: Sean ~~Larsen~~, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann,
Örs Tomaj Jeney, Dávid Borka

- We went through all open pull requests and discussed requested changes.
- We completed missing pull requests.
- We figured out new tasks.
- We determined story point values using Story Point Poker.

Appendix Figure A1: Meeting Log 15

Meeting Log

Date and time: 04.04.2026 12:00

Location: SDU Alsion - J1.07

Attendees: Sean Larsen, Kristóf Ákos Koltai, Denis Nedzhibov, Igor Zimmermann, ~~Örs Tomaj Jeney~~, Dávid Borka, Riccardo Terrenzi

- Scrum Master figured out proofreading and testing tasks for the next sprint.
- We as a group figured out even more remaining tasks.
- We calculated overall story point scores for all members.
- We determined story point values for the tasks and distributed based on contributions so far.

Appendix Figure AJ: Meeting Log 16